

Observability Analysis: A New Topological Algorithm

Roberto R. Nucera
Member

Michel L. Gilles
Member

Systems Control
ABB Power Automation Inc.
3174 Porter Drive, Palo Alto, CA 94304

Abstract - This paper presents an efficient topological algorithm for observability analysis. The algorithm has its roots in combinatorial optimization and relies on the concept of augmenting sequences. The correctness of the proposed algorithm is established by the proofs provided in the Appendices. The performance of the new algorithm is analysed using two real-life networks. We comment on our experience with a numerical observability algorithm, which has been implemented in several control centers. Performance comparisons of the proposed topological algorithm with the numerical observability algorithm are also provided.

Keywords: State-estimation, observability, topological algorithms, augmenting sequences, combinatorial optimization.

1 Introduction

Observability analysis is a fundamental component of any real-time State Estimator. Efficient implementation of observability is necessary in order to achieve the overall requirements for state estimation in modern control centers.

The theory of network observability has evolved into two classes of algorithms: numerically based and topologically based. Krumpholz, Clements and Davis [1] set forth the mathematical foundations of the combinatorial approach. The work in [2] also contained several results regarding topological observability. It was recognized in [3] that the topological approach could also be formulated as a matroid intersection problem. An extension of the theory [1], which permits the determination of all observable islands was presented in [4]. In [5]-[6], Monticelli and Wu launched the idea of numerically based observability. This method was then extended to fit other solution methods for the least-squares problem [7]-[9]. One trademark of [1] and [5]-[6] is the blend of mathematical proofs and efficient algorithms. Later, in [10] a method based on a symbolic reduction of the Jacobian matrix was developed.

Numerical observability appeals to the background of the power industry. It allows the use of existing routines for sparse triangular decomposition and sparse vector methods. However, since it requires floating point arithmetic, it is susceptible to roundoff errors. The numerical method has been successfully used in several versions of production grade state estimators.

Topological observability, on the other hand, is free from accuracy problems and appeals to the attractive concept of measurement assignment. It is important to note that some reluctance has prevailed in the industry, regarding the implementation of topological observability algorithms.

The so called graph-theoretic methods have been labelled as complex and/or computationally expensive [5]-[7] and [10]. We believe that the existence of some very inefficient algorithms for topological observability has generated some doubts regarding the practicality of the topological method. In [9], a performance comparison between a numerical algorithm and a topological algorithm was provided. The unusually large CPU-times given for the topological algorithm would most certainly discourage any attempt to implement topological methods. To the best of our knowledge, the work hereby presented is the first to compare two optimized, production grade versions, of a numerical algorithm and a topological algorithm. In all tests, real-life power networks are used.

This paper provides the following contributions:

- A clear application of the concept of augmenting sequences [11] to the problem of topological observability. Step-by-step explanations of this concept are provided through several examples.
- An efficient and field-tested algorithm for topological observability, which uses the merging of flow islands to speed up computations.
- A direct comparison between the proposed topological algorithm and a numerical observability algorithm, using production grade versions of our state estimator.

The key idea behind any topological observability is the determination of a maximal forest of full rank [1]. There are numerous ways in which the determination of the observable forest can be implemented. This paper presents an efficient topological algorithm, based on the concept of augmenting sequences, which correctly determines a maximal forest of full rank. Proofs of the proposed algorithm are given in Appendix 2.

We also intend to show that, although somehow foreign to the power area, combinatorial algorithms should not be labelled as complex. The algorithm of this paper as well as the one of [1], use very well known graph theory concepts.

The results of our experiments indicate that topologically based algorithms are several times faster than the numerical ones. One of the reasons being that no floating point computations are performed. As the test results given in Section 10 show, the average CPU time used by the topological algorithm hereby presented is roughly identical to the average CPU time used in one single factorization of a positive-definite decoupled (real or reactive) gain matrix.

We believe that the algorithm of Krumpholz, Clements and Davis [1], if well implemented, can achieve similar performance, due to the fact that it also takes advantage of the non-trivial observation that injection measurements can be processed only once and in no particular order.

2 Preliminaries

Observability analysis is usually performed with the assumption that real and reactive measurements are available in pairs. In this paper,

90 SM 488-7 PWRS A paper recommended and approved by the IEEE Power System Engineering Committee of the IEEE Power Engineering Society for presentation at the IEEE/PES 1990 Summer Meeting, Minneapolis, Minnesota, July 15-19, 1990. Manuscript submitted September 1, 1990; made available for printing July 6 1990.

the measurement set to be considered consists only of paired flow and injection measurements. No distinction is made between injection measurements and zero injections; they are equivalent as far as observability is concerned. The term injection measurement is used for both “true” injection measurements and zero injections. A branch of the power system, connecting buses k and m , is denoted b_{km} . A flow measurement in branch b_{km} is denoted p_{km} . An injection measurement at bus k is denoted P_k . A tree T of a graph is a connected subset of the graph with no loops. A forest F of a graph is a set of node-disjoint trees. The span (also closure) of a forest (or tree) F , also $\text{sp}(F)$, is the set of all branches that either belong to F or form loops with the branches of F . The concept of span is used in Section 9 and in Appendix 2.

3 An Overview of Topological Observability

Topological algorithms, as defined in [1], are based on the condition that, given a power network N , a subnetwork N_k of N is solvable, if and only if, it is possible to find a tree of N containing all buses of N_k , in which there is a one to one correspondence between measurements and branches. This leads to the concept of maximal forest of full rank (or simply maximal forest). This idea must be well understood before entering a description of the algorithm. We will review the matter quite informally. For precise definitions and more details, please refer to [1].

3.1 The Measurement Assignment

Consider a one-line diagram X representing a power network. A measurement assignment is a function that assigns to flow measurements and injection measurements of the network one and only one branch of a forest F_x of X . In other words, measurements are assigned to branches of the network, such that *no loops are formed*, according to the following rules:

- Flow measurements, if assigned, must be assigned to the corresponding measured branch.
- Injection measurements, if assigned, are assigned exactly to one of its incident branches.

3.2 The Maximal Forest

The maximal forest of full rank or just maximal forest is the “largest” forest of the one-line that can be built through a measurement assignment. In other words, a maximal forest must contain the maximum number of branches (and buses) possible in any measurement assignment of the network. The maximal forest is not unique, however, this fact is of secondary importance. Once a maximal forest has been determined (the choice is arbitrary), the observable subnetworks can be uniquely identified.

It has been proven in [4] that determining the maximal forest must be the initial target of any topological algorithm. Informally, the problem is to assign the maximum possible number of measurements (flows and injections) to branches of the one-line as long as loops are not created.

This point of view defines a matroid intersection problem. Details of matroid intersection algorithms can be found in [11]. We will present the proposed observability algorithm without resorting to any specific definitions involving matroid intersection, except for the concept of proper augmenting sequences. The algorithm is simple and can be implemented very efficiently. It must be noted that the problem of observability is a somehow simplified example of graphic matroid intersection.

4 Visualizing the Measurement Assignment

In the following discussion we will use the network of Figure 1.

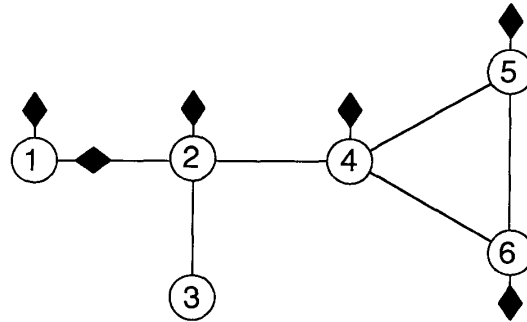


Figure 1: The 6-nodes network

In the measurement assignment function, flows have a unique assignment, which is the measured branch, while an injection may be assigned to any one of its incident branches. For example, in the network of Figure 1, flow measurement p_{12} must be assigned to branch b_{12} , whereas injection P_4 can be assigned to either b_{24} , b_{45} or b_{46} . In the same manner, injection P_6 can be assigned to branch b_{46} or to branch b_{56} .

Graph G , shown in Figure 2, can be used to compactly represent all assignment possibilities for each injection measurement.

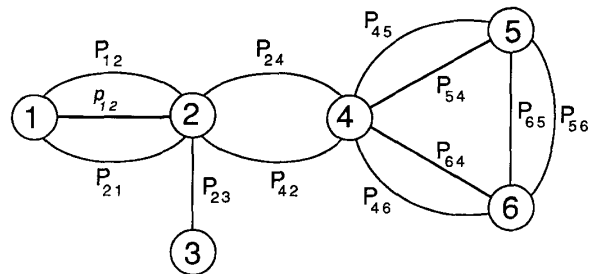


Figure 2: The component graph G for the 6-nodes network

At this point, we make the following remarks:

- Edge P_{23} , for example, is due to the possible assignment of injection P_2 to branch b_{23} , while edge P_{24} is due to the possible assignment of injection P_2 to branch b_{24} . We call P_{21} , P_{23} and P_{24} the *components* of injection P_2 in G . We shall refer to G as being the *component graph*.
- Since injection components are simply edges in graph G , the terms component and edge are used interchangeably throughout this paper. Also, since a given measurement assignment defines uniquely a forest of G , the terms forest and assignment are used interchangeably in the text.
- The process of growing a maximal forest can be seen as selecting the maximum possible number of edges of G , maintaining, at all times, the rules given in Section 3.

5 Augmenting Sequences

The algorithm described in this paper is based on the concept of augmenting sequences [11]. Without giving any definition at this point, we shall provide an example of the determination of an augmenting sequence, using the network of Figure 1. In this manner, the concept will be informally introduced. A mathematically precise definition of augmenting sequences is provided in Appendix 1.

Considering the network of Figure 1, we shall assume that, at some step of the algorithm, we have the assignment given in Figure 3 below.

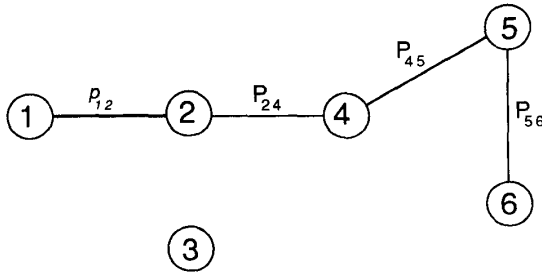


Figure 3: Initial assignment or forest

Note that flow p_{12} is assigned to its measured branch. Injection P_2 is assigned via component P_{24} , i.e., P_2 is assigned to branch b_{24} , while P_4 is assigned via P_{45} and injection P_5 is assigned via P_{56} . Injection P_6 has not been processed as yet.

The measurement assignment is in accordance to all rules set forth in Section 3, however, it is not maximum. At this point, it is obvious that we must attempt to assign injection P_6 so as to increase the number of branches in the forest by one, and therefore, reach bus 3. Clearly, the expansion of the forest must be achieved without violating the rules of the measurement assignment function.

Initially, we must determine what options are available. Referring to Figure 2, we note that injection P_6 can be assigned (included in the forest) via either P_{64} or P_{65} . However, inspection of Figure 3, reveals that component P_{64} forms a loop with edges P_{45} and P_{56} (of the current forest). Thus, if we desire to assign P_6 via P_{64} , we must remove either one of edges P_{45} or P_{56} from the current assignment. In this manner, the loop is broken and the legality of the assignment function is maintained. Likewise, if we were to assign P_6 via P_{65} , a loop consisting of edges P_{56} and P_{65} would be formed, and therefore, edge P_{56} would have to be removed from the forest. These possibilities are displayed by the arrows that lead to level number 2 of the *level (or border) graph* of Figure 4.

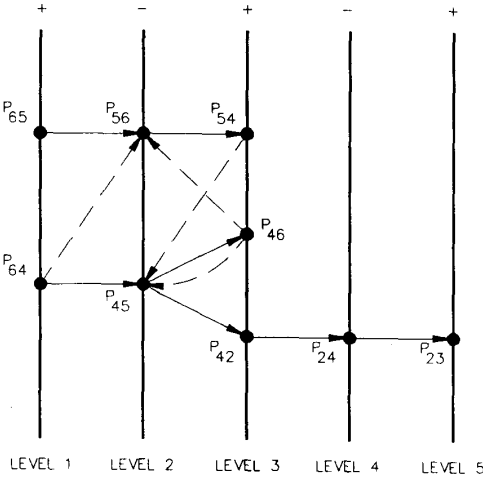


Figure 4: Border graph for the 6-nodes network

Let us examine Figure 3, which shows the current forest, in more detail. The removal of edge P_{45} , following the addition of edge P_{64} , would result in the the forest of Figure 5.

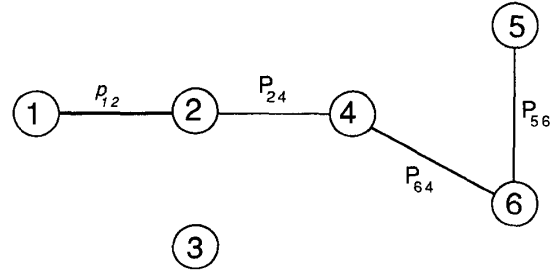


Figure 5: Addition of P_{64} and removal of P_{45}

Nothing was gained, since we managed to include injection P_6 in the assignment, but we "lost" injection P_4 and the number of branches of the forest has not been increased. It is logical that we attempt to reassign injection P_4 via some component, other than P_{45} . Referring to Figure 2, the available choices are P_{46} and P_{42} . This is shown by the two solid arrows that originate from P_{45} in the border graph of Figure 4, leading to the third level.

Continuing the same reasoning, from the third level on, the addition of P_{42} to the forest of Figure 3 would create a loop, involving edges P_{42} and P_{24} . If we want to add P_{42} we must, therefore, remove P_{24} . The updated forest would be given in Figure 6.

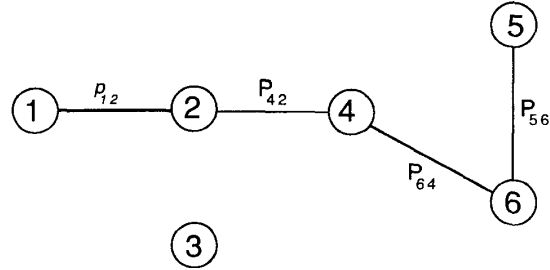


Figure 6: Addition of P_{42} and removal of P_{24}

Thus, injection P_2 was deassigned. This is shown by the solid arrow reaching level 4 in the border graph of Figure 4. Since we have now "lost" injection P_2 , we try to reassign it. The only choice for reassignment is P_{23} . Referring to Figure 3, we see that no loops are formed by the addition of P_{23} and, therefore, injection P_6 has been successfully included in the assignment. We say successfully, because no injection previously assigned had to be removed and the forest has been increased by one branch. The updated forest, which is now maximal, is shown in Figure 7.

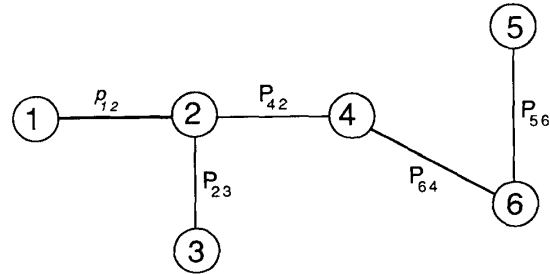


Figure 7: Maximal forest for the 6-nodes network

At this point, several important observations can be made:

- The assignment of injection P_6 was possible due to the following sequence of operations performed in the forest of Figure 3.

Addition of edge P_{64} , removal of edge P_{45} , addition of edge P_{42} , removal of edge P_{24} and addition of edge P_{23} . This defines the *augmenting sequence* $[P_{64}, P_{45}, P_{42}, P_{24}, P_{23}]$.

- An augmenting sequence can be visualized in levels as follows: at each odd level, an edge of G , Figure 2 in the above example, not in the current forest F , is added to F , perhaps creating a unique loop. This loop is then broken at the next even level by the removal of some other edge of F , and so forth, until we can add a branch without forming any loop. The plus and minus signs on top of each level, in Figure 4, indicate insertion and removal, respectively.
- The number of edges in the sequence just obtained is odd. Thus, the number of branches in the forest is increased by one. However, our search in the border graph may terminate with a sequence containing an even number of edges. In this case, the sequence is not augmenting and the injection being processed is *redundant*. This fact is justified by Theorem 2 in Appendix 2. Note that if the number of edges in a sequence is even, the number of edges to be added equals the number of edges to be removed and the forest cannot be augmented.
- In the border graph of Figure 4, we show all possibilities available at each step. The broken lines indicate paths that would lead to already visited nodes of the border graph. Since we never visit nodes already “seen”, the search procedure is of “breadth-first” type (for details, please refer to [11]). It follows automatically that the augmenting sequences found in this manner admit no shortcuts. We shall refer to this “shortest” augmenting sequences as *proper* augmenting sequences. There may be longer augmenting sequences; however, we do not need to bother with them. Theorem 3, in Appendix 2, proves that the search for proper augmenting sequences is sufficient to achieve optimality.

6 Processing Flow Measurements

In the example given above, no distinction was made between flow measurements and injection measurements. Indeed, in the graph G of Figure 2 we include flow measurement p_{12} .

However, it is easy to see that there is no need to carry flow measurements in graph G , since their assignment is unique. This would be wasteful; instead we merge all buses “connected” by flow measurements into a single node or *superbus*. For example buses 1 and 2 could have been merged into a single node in the graph G of Figure 2. The same idea was proposed in [2].

The processing of flow measurements can be done extremely quickly by a “greedy” type algorithm. Buses joined by flow measurements, forming what we call *flow islands*, are merged in a single node in graph G .

The merging of flow islands into superbuses will, in most cases, reduce considerably the size of the graphs involved, enhancing the performance of the algorithm.

7 Description of the Algorithm to Determine a Maximal Forest

The algorithm, informally introduced in the preceeding sections, can be described by the following three major steps:

- Step 1.** Process all available flow measurements, merging buses “connected” by flow measurements (flow islands) into superbuses.
- Step 2.** Build graph G of injection components. Note that flow measurements do not appear in G .

Step 3. Process all available injection measurements. This is done one injection at a time (in no particular order). For each injection P_k , attempt to find an augmenting sequence. If an augmenting sequence is found, **update the current forest** to reflect the augmenting sequence, else, **update the list of redundant injections** to include P_k .

The algorithm processes injections sequentially in an attempt to enlarge the forest. Each injection that is examined is either used to add a branch to the forest or is labelled as redundant with respect to the set of measurements already processed (including flow measurements).

The maximal forest determined by the algorithm described above is dependent on the order in which injections are processed, as well as the order of the edges in the adjacency lists used to store the graphs.

After a maximal forest has been obtained it is necessary to determine the observable subnetworks (islands) that can be uniquely solved by the state estimator. This second phase of the observability algorithm is addressed in Section 9.

8 Example

We now provide a complete and detailed example of the determination of a maximal forest. The example will continue in Section 9, when the observable subnetworks are identified. The 10-nodes network of Figure 8 will be considered throughout this section.

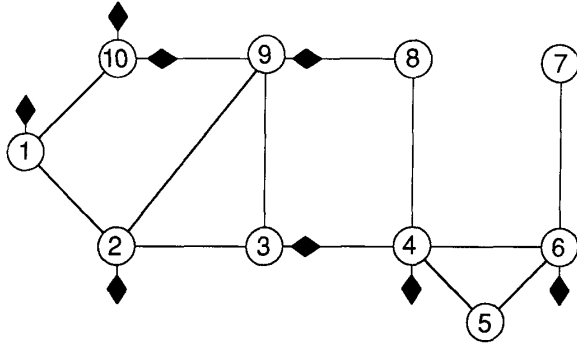


Figure 8: The 10-nodes network

Step 1.

Flow measurement p_{34} joins buses 3 and 4, resulting in superbus 3 (flow islands are identified by their lowered number bus), while p_{89} and p_{9-10} join buses 8, 9 and 10 into superbus 8.

Step 2.

The graph G of components is shown in Figure 9. It is important to note that although some buses have lost their individuality in Step 1, the components still refer to the original bus numbers. Also, some components have been eliminated. For example, P_{43} is not in G due to the merging of buses 3 and 4.

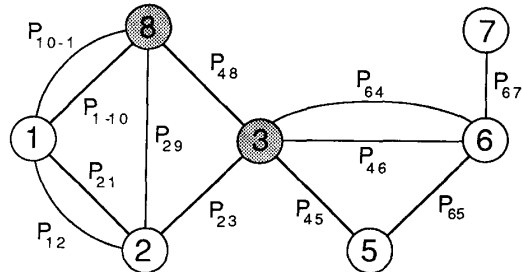


Figure 9: The component graph G for the 10-nodes network

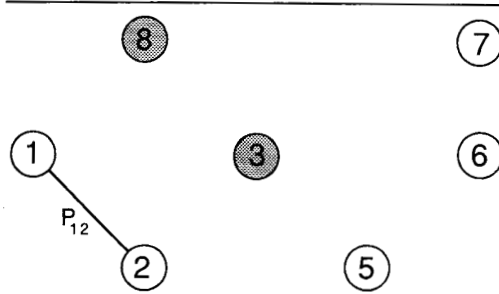
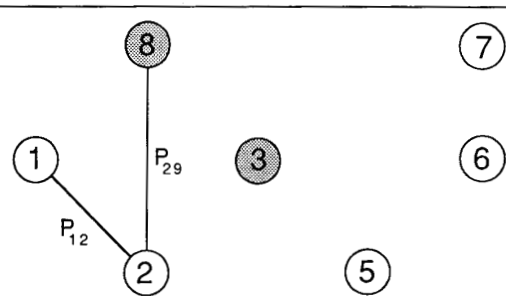
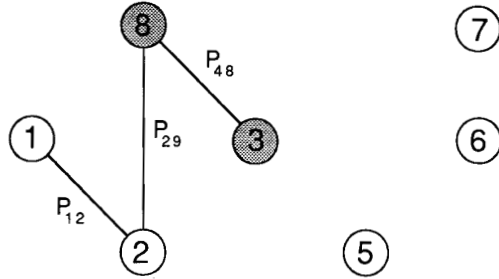
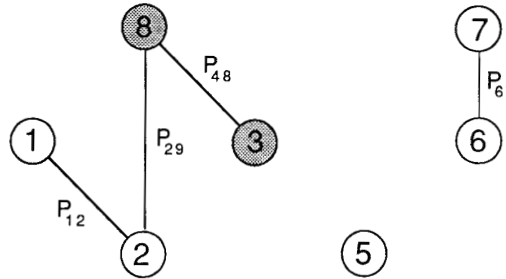
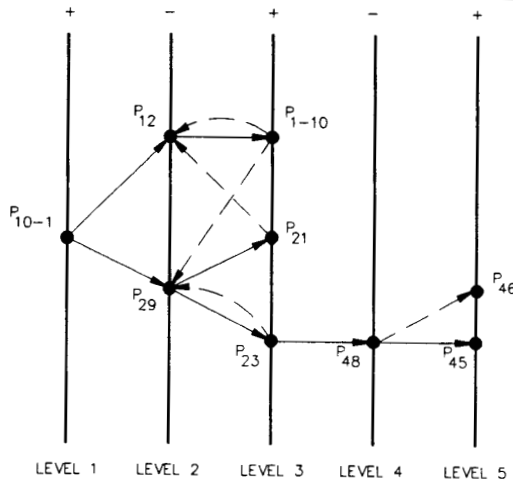
Figure 10: Forest after the assignment of injection P_1 Figure 11: Forest after the assignment of injection P_2 Figure 12: Forest after the assignment of injection P_4 Figure 13: Forest after the assignment of injection P_6 

Figure 14: Border graph of the 10-nodes network

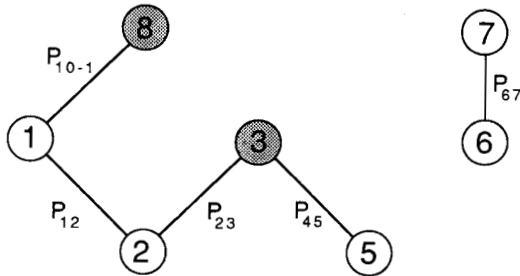


Figure 15: Maximal forest for the 10-nodes network

Step 3.

1. Injection P_1 can be assigned via P_{12} or P_{1-10} , none of which form loops in the current forest. Arbitrarily, we select edge P_{12} . The augmenting sequence is $[P_{12}]$; the updated forest is shown in Figure 10.
2. Injection P_2 can be assigned via P_{21} , P_{23} or P_{29} . However, as shown in Figure 10, P_{21} would form a loop with the already assigned P_{12} . Therefore, we must choose between P_{23} and P_{29} ; arbitrarily, we select P_{29} . The augmenting sequence is $[P_{29}]$; the updated forest is shown in Figure 11.
3. Injection P_4 can be assigned via P_{45} , P_{46} or P_{48} , none forming loops in the current forest. Arbitrarily, we choose edge P_{48} . The augmenting sequence is $[P_{48}]$; the updated forest is given in Figure 12.
4. Injection P_6 can be assigned via P_{64} , P_{65} or P_{67} , none of which form loops in the current forest. Arbitrarily, we select edge P_{67} . The augmenting sequence is $[P_{67}]$; the updated forest is given in Figure 13.
5. Injection P_{10} can only be assigned via component P_{10-1} . However, the addition of P_{10-1} to the forest of Figure 13, would create a loop, involving edges P_{12} and P_{29} . Clearly, we need a border graph, which is given in full detail in Figure 14. The augmenting sequence found is given by $[P_{10-1}, P_{29}, P_{23}, P_{48}, P_{45}]$. The final and maximal forest is shown in Figure 15.

Once more, we must note that the broken arrows in Figure 14 are superfluous and that our search is of breadth-first type. Note that we assume, in the last step, that P_{45} appears first than P_{46} in the adjacency lists for bus 4. Thus, the broken arrow from P_{48} to P_{46} in the border graph.

In passing, it should be mentioned that augmenting sequences are not unique. Depending on the data structures used, a perfectly correct and different augmenting sequence could have been constructed. This, in accordance to the fact that the maximal forest itself is not unique.

9 Identification of Observable Islands

The maximal forest of full rank is not unique. However, the maximal observable subnetwork [1], i.e., the maximum number of observable buses and, therefore, observable islands is guaranteed to be unique. The procedure to obtain the maximal observable subnetwork has been very well explained (for details, please refer to [4]).

The fundamental rule in this important stage of the observability process is as follows: assigned injections associated to a bus that has, at least one incident branch not in the span of the maximal forest, cannot be used in the solution and must be removed from the forest. Since the removal of an injection will partition the corresponding tree, its nodes must be revisited until no more unusable injections remain.

Referring to the maximal forest given in Figure 15, bus 4 has incident branches not in the span of the forest. Note that branch b_{46} , of the 10-nodes network, does not form loops with the branches defined by the forest of Figure 15. Thus, injection P_4 must be deassigned. The same applies to injection P_6 , since b_{56} and b_{46} are not in the span of the forest. Therefore, the output of the observability algorithm would identify one observable island, formed by buses 1, 2, 3, 4, 8, 9, 10. Buses 5, 6, 7 are unobservable.

It is worthwhile to mention that, with real-life systems, the number of unusable injections may be quite large [8].

10 Performance Considerations - Comparison with a Numerical Observability Algorithm

In this section we discuss, in detail, the performance of the proposed topological algorithm. We also provide comparisons with a field tested numerical observability algorithm, including some remarks on our experience with the numerical approach. In all tests, both methods generated exactly the same output results, verifying the correctness of the algorithms used.

The implementation of the numerical approach follows very closely the steps of the algorithm given in [6]. The only difference that needs to be mentioned is that, in Step 5 of the algorithm given in [6], the DC state estimator equation is solved using fast-forward substitution, due to the sparsity of the right-hand side. Also, we have verified that the numerical approach requires special logic with 32-bit word-length [8], due to the necessity of precisely detecting zero pivots. The main problem posed by the word-length is the increase in the memory required to store the factors of the observability gain matrix.

Some characteristic results obtained with two real-life power systems are provided. The systems have distinct measurement sets. System 1 has a high proportion of injection measurements relatively to the number of flow measurements. In its standard measurement configuration it is mostly observable. System 2 is characterized by a very large external model; in its standard configuration, roughly 2/3 of its buses are unobservable. It has more flow measurements than injection measurements.

In the tables provided below, the configuration of Case 1 is the standard measurement availability. Case 2 was obtained by disabling all flow measurements (all injections case), while in Case 3, all injections, including zero injections, were disabled (all flows case).

An island, to be considered as such, must contain a minimum of two buses. The results of the observability analysis are given without regards to the existence of voltage measurements. Thus, the number of islands does not reflect the fact that some islands do not contain voltage measurements and, therefore, will not be used in the solution process. For instance, of the 16 islands found in Case 1 of Table 1, only 6 islands, totaling 458 buses, are used by the state estimator. In Case 1 of Table 2, only 1 of the 22 islands is used. This island consisted of 532 buses. Generally, islands without voltage measurements are very small.

Each case, per table, contains the measurement configuration used and the output results obtained by the observability algorithms. In the "CPU" column we provide the CPU times (in seconds) used by each algorithm. "Ratio" is the smallest execution time normalized by the largest of the two execution times. In all cases tested, the topological algorithm was faster. "Factor." refers to the time used to factorize the *decoupled real* gain matrix. All CPU times were taken using a VAX 8700. The factorization time must be used with caution. If the number of unusable injections is large, the dimensions of the system change considerably and any comparison is meaningless (please note Case 2 in both tables below).

TABLE 1 - OBSERVABILITY ANALYSIS: SYSTEM 1

Case 1			
Configuration	Results	CPU	
594 buses	16 islands	Numerical	=> 0.44
427 injections	505 obs. buses	Topological	=> 0.13
357 flows	22 unused inj.	Ratio	=> 0.29
		Factor.	=> 0.11

Case 2			
Configuration	Results	CPU	
594 buses	54 islands	Numerical	=> 0.46
427 injections	221 obs. buses	Topological	=> 0.11
0 flows	128 unused inj.	Ratio	=> 0.24
		Factor.	=> 0.05

Case 3			
Configuration	Results	CPU	
594 buses	18 islands	Numerical	=> 0.11
0 injections	236 obs. buses	Topological	=> 0.07
357 flows	0 unused inj.	Ratio	=> 0.62
		Factor.	=> 0.06

The high performance of the topological algorithm must be noted. The topological algorithm, in Case 1 of Table 1, determined 18 flow islands, one of which contained 168 buses. The longest augmenting sequence found in Case 1 had 3 edges. For Case 2 of Table 1, the longest augmenting sequence had 5 edges. This shows that augmenting sequences are, generally, very short and tend to decrease as the number of flow measurements increases, since the graph becomes denser. The longest loop encountered in the process of finding augmenting sequences for the cases of Table 1, contained 7 edges, which occurred in Case 2. Thus, in the vast majority of cases augmenting sequences are found with very little search.

It is also noticeable that the numerical algorithm is more sensitive to unusable injections than the topological one. The following observations, regarding the numerical approach, account for this fact:

- If the number of injections to be removed is significant, it is more economical to rebuild the gain matrix than to perform updates.
- There are cases in which more than one iteration is necessary to eliminate all unusable injections. These cases are rare, but do happen. Indeed, Case 2 above required three factorizations of the gain matrix.

On the other hand, revisiting the nodes of the observable forest, in order to remove unusable injections, can be done with very little effort.

TABLE II - OBSERVABILITY ANALYSIS: SYSTEM 2

Case 1			
Configuration	Results	CPU	
1482 buses	22 islands	Numerical	=>0.71
354 injections	601 obs. buses	Topological	=>0.20
572 flows	10 unused inj.	Ratio	=>0.28
		Factor.	=>0.16
Case 2			
Configuration	Results	CPU	
1482 buses	56 islands	Numerical	=>0.86
354 injections	221 obs. buses	Topological	=>0.22
0 flows	189 unused inj.	Ratio	=>0.26
		Factor.	=>0.05
Case 3			
Configuration	Results	CPU	
1482 buses	9 islands	Numerical	=>0.31
0 injections	458 obs. buses	Topological	=>0.10
572 flows	0 unused inj.	Ratio	=>0.32
		Factor.	=>0.10

The topological algorithm, in Case 1 of Table 2, found 9 flow islands, one of which contained 432 buses. The maximum length of any augmenting sequence found in Case 1 of Table 2 was 3, while for Case 2 of Table 2, it was 7. The maximum number of edges in any loop encountered in the process of determining augmenting sequences was 3, which happened in Case 2 (Table 2). Again the numerical algorithm, showed to be more sensitive to the number of unusable injections than the topological algorithm.

11 Conclusions

An efficient algorithm for topological observability has been presented. Its correctness is rigorously established by the proofs provided in Appendix 2. The algorithm has its roots in combinatorial optimization and was presented with the use of simple and well-known concepts from graph theory. The concept of augmenting sequences was clearly applied to topological observability. This paper should contribute to the understanding of the topological method and should provide guidelines for its efficient implementation.

A direct comparison between the performance of a topological algorithm and the performance of a numerical algorithm has been provided. Both algorithms have very optimized implementation and are integral part of different versions of production grade state estimators. The tests were designed to test both algorithms under extreme conditions and with a variety of measurement configurations. Results obtained with two real-life power systems have been provided.

The experiments conducted using our algorithms indicate that the topological approach seems to have much better performance than the numerical method. The complete results of the topological observability analysis can be obtained, approximately, in the same time necessary to factorize the real decoupled gain matrix used in the subsequent solution process.

Acknowledgments

The authors are grateful to Prof. Kevin Clements for reviewing an earlier draft of this paper. His comments certainly added value to this work.

References

- [1] G. R. Krumpholz, K. A. Clements and P. W. Davis, "Power system observability: a practical algorithm using network topology", *IEEE Trans. Power Apparatus and Systems*, Vol. PAS-99, No. 4, pp. 1534-1542, July/Aug. 1980.
- [2] J. J. Allemong, G. D. Irisari and A. M. Sasson, "An examination of solvability for state estimation algorithms", Paper A80008-3, *IEEE PES Winter Meeting*, New York, N.Y., 1980.
- [3] V. H. Quintana, A. Simoes-Costa and A. Mandel, "Power system observability using a direct graph-theoretic approach", *IEEE Trans. Power Apparatus and Systems*, Vol. PAS-101, No. 3, pp. 617-626, March 1982.
- [4] K. A. Clements, G. R. Krumpholz and P. M. Davis, "Power system state estimation with measurement deficiency: an algorithm that determines the maximal observable subnetwork", *IEEE Trans. Power Apparatus and Systems*, Vol. PAS-101, No. 9, pp. 3044-3052, Sept. 1982.
- [5] A. Monticelli and F. F. Wu, "Network observability: theory", *IEEE Trans. Power Apparatus and Systems*, Vol. PAS-104, No. 5, pp. 1042-1048, May 1985.
- [6] A. Monticelli and F. F. Wu, "Network observability: identification of observable islands and measurement placement", *IEEE Trans. Power Apparatus and Systems*, Vol. PAS-104, No. 5, pp. 1035-1041, May 1985.
- [7] A. Monticelli and F. F. Wu, "Observability analysis for orthogonal transformation based state estimation", *IEEE Trans. on Power Systems*, Vol. PWRS-1, No. 1, pp. 201-208, February 1986.
- [8] F. F. Wu, W.-H. E. Liu and S.-M. Lun, "Observability analysis and bad data processing for state estimation with equality constraints", *IEEE Trans. on Power Systems*, Vol. 3, No. 2, pp. 541-548, May 1988.
- [9] F. F. Wu, W.-H. E. Liu, L. Holten, A. Gjelsvik and S. Aam, "Observability analysis and bad data processing for state estimation using Hachtel's augmented matrix method", *IEEE Trans. on Power Systems*, Vol. 3, No. 2, pp. 604-611, May 1988.
- [10] I. W. Slutsker and J. M. Scudder, "Network observability analysis through measurement jacobian matrix reduction", *IEEE Trans. on Power Systems*, Vol. PWRS-2, No. 2, pp. 331-338, May 1987.
- [11] C. H. Papadimitriou, K. Steiglitz, *Combinatorial Optimization*, Prentice Hall, Inc., 1982.

Appendix 1

This appendix provides the precise mathematical definitions necessary to complement the proofs of Appendix 2.

In the two Appendices, we shall use the abbreviation e_i to represent some edge i of a graph. Clearly, each edge e_i represents uniquely some injection component in graphs G and F .

Although not necessary in the main text, we need here to define the concept of an alternating (not necessarily proper) sequence.

In the Appendices, if S and T are sets, then $S \oplus T$ represents their symmetric difference [11]. Also, a *free* component refers to an injection component not assigned in the current forest.

Definition 1.1

A sequence $S = [e_1, e_2, \dots, e_m]$ is called *alternating* with respect to an assignment F if:

- A1) e_i is a component of an unassigned injection;
 A2) For all even i , $2 \leq i \leq m$, $e_i \in F$, and $\text{sp}(F \oplus S_i) = \text{sp}(F)$;
 A3) For all odd i , $3 \leq i \leq m$, $e_i \notin F$, besides, if $e_{i-1} = P_{uv}$, then e_i is also a component of P_u .

The algorithm presented in Section 7 uses alternating sequences that admit no shortcuts. These are called proper sequences. Before defining proper sequences precisely, we need the following convention. Assume that, given an assignment F , the insertion of a new component or edge e_i creates a loop in F . We shall call this unique loop C_i .

Definition 1.2

A sequence $S = [e_1, e_2, \dots, e_m]$ is *proper* with respect to an assignment F if:

- B1) e_1 is a component of an unassigned injection;
 B2) For all even i , $2 \leq i \leq m$, $e_i \in F$ and $e_i \in C_{i-1}$. Furthermore, $e_i \notin C_{k-1}$ for any even $k < i$;
 B3) For all odd i , $3 \leq i \leq m$, $e_i \notin F$, besides, if $e_{i-1} = P_{uv}$, then e_i is also a component of P_u . Furthermore, P_u is used only once in the sequence.

Appendix 2

Theorem 1. Any correct breadth-first search procedure that fans out from the components of an unassigned injection P_u , in which each even-level labels elements associated with free components of the current injection and each odd-level element e_i labels elements associated with edges of C_i , will find a proper augmenting sequence from P_u , if such sequence exists.

Proof. Assume that there exists one proper augmenting sequence $S = [e_1, e_2, \dots, e_m]$. Considering Definition 1.2, by B1, e_1 is a free component of P_u . This element is certainly in the first level of the graph. Using B2, e_2 is an edge of C_1 ; certainly, there is an arrow (e_1, e_2) in the level graph. Now, if $e_2 = P_k$ then, by B3, e_3 is a free component of P_k . Certainly, there is an arrow (e_2, e_3) in the level graph, and so on, until e_m is reached. Thus, there is a path between e_1 and e_m . Since we proceed in breadth-first manner, we move along shortest paths (admitting no shortcuts) and therefore all sequences found are proper. //

Theorem 2. Suppose that there is no augmenting sequence in F from an unassigned injection measurement P_u . Let S_w be an augmenting sequence in F from some other unassigned injection P_w . Then, there is no augmenting sequence from P_u in $F \oplus S_w$ either.

Proof. Assume there is an augmenting sequence S_u from P_u in $F \oplus S_w$. Suppose no even level edge of S_w was in any loop C_i of S_u . Then, the augmentation from S_w changes nothing as far as S_u is concerned, and thus, S_u was an augmenting sequence from P_u prior to the augmentation, contradicting the assumption. So, let C_i be the loop in S_u which contains an edge e_m of S_w . Then, we could choose e_m as the next even level edge to continue S_u and proceed along the remaining portion of S_w . Again, we would have an augmenting sequence from P_u prior to the augmentation from P_w , contrary to the assumption. //

Corollary 1. If at some stage of the algorithm, there is no augmenting sequence from an unassigned injection P_u , then there will be no stage of the algorithm in which there is an augmenting sequence with respect to P_u .

Proof. By induction on n , for all n , n augmentations after we failed to discover an augmenting sequence from P_u , there will still be no augmenting sequence from P_u .

Theorem 3. A measurement assignment F of a network N is maximum, if and only if, there is no proper augmenting sequence in F from any unassigned injection measurement of N .

Proof. It is clear that if an assignment F admits an augmenting sequence, then the assignment is not maximum. For the converse, suppose there is no proper augmenting sequence in F , and yet there is an assignment J of cardinality $|J| = |F| + 1$. Assume that there is an unassigned injection which has a component not in the span of any tree of F . Then, this constitutes an augmenting sequence in F , contradicting the assumption. Thus, let any unassigned injection have all components in the span of a tree of F , which implies that $\text{sp}(F)$ is augmented by an edge P_{uv} , which is a free component of some injection P_u assigned in F . Let T be the tree of F which contains P_u . In order to include P_{uv} in the forest, we must remove an edge P_{uw} of P_u which is in F . The assumption that $|J| = |F| + 1$, implies that the span of T can be restored. Let P_{km} be the edge of $J - F$ that restores the span. If P_k is not in F , we have an augmenting sequence $[P_{km}, P_{uw}, P_{uv}]$. Otherwise, we must remove the component of P_k which is in F , and find some other edge of $J - F$ to restore $\text{sp}(T)$. We continue in this manner, until the edge of $J - F$ used to restore the span is a component of an unassigned injection (injection not in F). We know that there is such component by assumption; clearly, we have just built an augmenting sequence. Finally, if there is an augmenting sequence, then there exists an augmenting sequence that admits no shortcuts. This constitutes a proper augmenting sequence, contradicting our assumption. //

Biographies

Roberto R. Nucera (Member, IEEE) received the B.S. from the Catholic University of Rio de Janeiro, Brazil, in 1983 and the M.S. from Purdue University in 1985. He has been with Systems Control, Inc. since 1988, working with the State Estimator software. His primary areas of interest are power systems analysis and optimization.

Michel L. Gilles (Member IEEE) received the Legal Degree of Electro-Mechanical Civil Engineer, major in Electricity, from the University of Liege, Belgium, in 1977 and the M.S.E.E. and Ph.D. degrees in 1979 and 1981 respectively, both from Wayne State University. He is presently the manager of the Network Analysis Group at Systems Control, Inc.. His primary areas of interest are power systems analysis and optimization.

Discussion

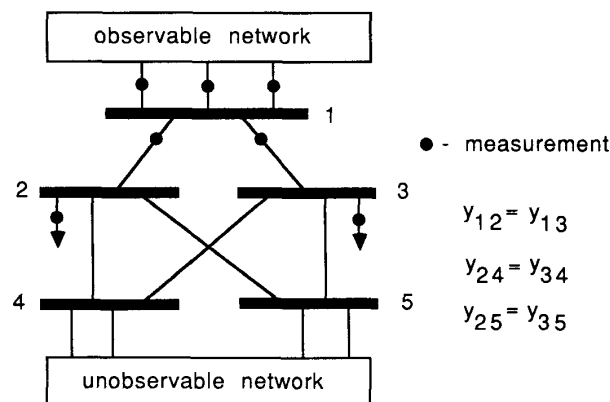
I. W. Slutsker (Empros Systems International, Plymouth, Minnesota) and **A. Monticelli** (UNICAMP, Brazil): The authors are commended on a development of an improved topological observability method which belongs to a general class of non-numeric observability methods.

Non-numeric methods, whether topological, combinatorial or symbolic, identify network observability by analyzing the information about network and measurement topology contained in the measurement Jacobian matrix without taking into account the actual values of matrix elements. Some implementations of non-numeric methods appear fast enough for on-line use, although the amount of overhead and implementation complexity of some of the more intricate schemes may be substantial.

The drawback of non-numeric methods is their inherent inability to detect numerical instability or outright singularity of the solution matrices for networks that are otherwise topologically observable.

Occasionally, this may be caused by a particular combination of network impedances or weighting factors and can be alleviated by changes in the data base without significantly compromising the solution accuracy. In some cases the ill-conditioning is not too severe and can be overcome by the application of a more robust solution algorithm following the observability analysis.

However, there are situations in which the topologically observable network yields a strictly singular gain matrix. One such case was observed in the network of a northern California utility. A parallel connection of the two three winding transformers leads to the following network



In this case buses 4 and 5 are perfectly topologically observable but the gain matrix is singular. Even a fully coupled formulation fails here, since state variables at buses 2 and 3 tend to be equal as well as variables at buses 4 and 5. When such a case is encountered in the field the speed advantage of non-numeric methods no longer applies. In fact, non-numeric methods become slower as additional factorizations are necessary to weed out the singularity.

The only way to correct a situation like this if a non-numeric observability method is employed is to ask utility personnel to change network configuration (i.e. add a small impedance branch) or to add pseudo measurements. It is always embarrassing to ask users to change their model to compensate for the deficiencies of the algorithm.

It is ironic that the more robust the non-numeric method is, the farther it extends the non-numeric observability into numerically marginally observable portions of the network, the better are its chances of encountering the numerical instability of the solution process.

The numerical observability methods, if used properly, can be quite advantageous. Once the unobservable (boundary) injections and stand alone measurements are identified and removed at almost no computational cost, the remaining measurement set is largely observable. There is no reason to perform a separate observability analysis. The factorization process can proceed as usual and when the unobservability is encountered, a critical pseudo measurement can be inserted to allow the factorization to be completed. Once the factorization is done, all measurements associated with the unobservable variables (they should be few in number in most cases) can be removed by the additional processing (this can be especially easily done if Givens rotations are employed, since the same factorization routine can be used). The overhead of that additional processing should be

quite competitive with the time required to perform a non-numeric observability analysis and the resulting factored matrix will guarantee a stable solution of the state estimation.

Finally, we should also mention the difficulty of maintaining a separate observability module by utility personnel, especially, for more exotic non-numeric schemes.

The authors' comments will be appreciated.

Manuscript received August 8, 1990.

R. NUCERA and M. GILLES: We thank the discussers for their interesting discussion, which addresses the need of monitoring the pivots during elimination of the gain matrices.

This paper provides a comparison between optimized versions of a topological algorithm and a numerical algorithm for observability analysis. The performance comparisons are given to illustrate the fact that topological algorithms, contrary to previous speculations, references [5-7,10] of the paper, can be very efficient. Speed is certainly a bonus in real-time implementations.

No matter what precautions are taken prior to factorization to predict a stable elimination, they may not be sufficient to overcome numerical difficulties. This is well known and by no means prevents or discourages the use of topological observability as the preparatory step for factorization.

In our implementation of the Blocked Augmented Method [A,B], we use a delayed elimination scheme. The number of delays is extremely small when using blocked factors [A]. At the end of factorization, it is possible that a very small number of rows could not be eliminated. The buses associated with rejected rows are dummied out of the matrix "on the fly" with no computational effort. Any measurements associated with these numerically unobservable buses are then removed by extremely fast factor updating techniques.

Most of the work of determining the observable buses is being performed by the topological method. Thus, we can guarantee that the number of numerical singularities, if any, will be very small.

Since the actual cause of the singularity is immaterial, the discussers' example becomes somewhat academic. Independently of the observability method used, it may be necessary to automatically reduce the size of the observable network to handle special numerical situations.

As far as viability of the discussers' proposed scheme to speed up the numerical approach we have serious reservations:

i) The discussers point out the fact that once unobservable boundary injections are removed, the remaining measurement set is "largely observable". But how are unobservable injections identified? Clearly, some type of non-numerical (topological) analysis must be performed in order to identify these injections.

ii) The discussers propose the use of "critical pseudomeasurements" to carry out factorization, as suggested by Alvarado and Tinney in [B]. As pointed out in [B], such schemes can only be efficient if the number of such pseudomeasurements is small. Again, some type of topological analysis must precede factorization.

iii) The discussers seem to neglect the iterative nature of observability algorithms. As indicated in Section 10 of the paper, the numerical algorithm may require three (3) factorizations in order to prune out all unusable injections. This numerical backtracking is very time consuming.

The above discussion shows that a topological analysis is desirable as a preparatory step for factorization. Based on our experience, with both topological and numerical observability methods, we believe that future developments should focus on a rapid and exact topological assessment of observability, followed by a robust solution method, with capability to filter out the few, if any, occurrences of numerical difficulties. With

this scheme, which guarantees a stable elimination, the need for numerical backtracking is practically eliminated.

[A] R. Nucera and M. Gilles, "A Blocked Sparse Formulation for the Solution of Equality-Constrained State Estimation", paper 90 WM 234-5 PWRS, IEEE/PES 1990 Winter Meeting, Atlanta, GA.

[B] F. Alvarado and W. Tinney, "State Estimation Using Augmented Blocked Matrices", Paper 90WM 241-0 PWRS, IEEE/PES 1990 Winter Meeting, Atlanta, GA.

Manuscript received November 9, 1990.